```
import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
import pickle
import tensorflow as tf
from tensorflow.keras.models import Model, load_model
from tensorflow.keras.layers import Input, Dense

from google.colab import drive


# ========================= DATA LOADING =========================
drive.mount("/content/drive", force_remount=True)
text_df = pd.read_csv("/content/drive/MyDrive/dataset/MentalHealth/depression_dataset_reddit_cleaned.csv", encoding="utf-8")
tab_df  = pd.read_csv("/content/drive/MyDrive/dataset/MentalHealth/Student Mental health.csv", encoding="utf-8")

print(text_df.head())
print(tab_df.head())
```

```
Mounted at /content/drive
                                          clean_text  is_depression
0  we understand that most people who reply immed...              1
1  welcome to r depression s check in post a plac...              1
2  anyone else instead of sleeping more when depr...              1
3  i ve kind of stuffed around a lot in my life d...              1
4  sleep is my greatest and most comforting escap...              1
        Timestamp Choose your gender   Age What is your course?  \
0  8/7/2020 12:02             Female  18.0          Engineering
1  8/7/2020 12:04               Male  21.0    Islamic education
2  8/7/2020 12:05               Male  19.0                  BIT
3  8/7/2020 12:06             Female  22.0                 Laws
4  8/7/2020 12:13               Male  23.0         Mathemathics

  Your current year of Study What is your CGPA? Marital status  \
0                     year 1        3.00 - 3.49             No
1                     year 2        3.00 - 3.49             No
2                     Year 1        3.00 - 3.49             No
3                     year 3        3.00 - 3.49            Yes
4                     year 4        3.00 - 3.49             No

  Do you have Depression? Do you have Anxiety? Do you have Panic attack?  \
0                     Yes                   No                        Yes
1                      No                  Yes                         No
2                     Yes                  Yes                        Yes
3                     Yes                   No                         No
4                      No                   No                         No

  Did you seek any specialist for a treatment?
0                                           No
1                                           No
2                                           No
3                                           No
4                                           No
```

```
# ========================= TEXT PREPROCESSING =========================
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+|www\S+', ' ', text)
    text = re.sub(r'@\w+', ' ', text)
    text = re.sub(r'#\w+', ' ', text)
    text = re.sub(r'[^a-zA-Z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

text_df = text_df[['clean_text', 'is_depression']].copy()
text_df['clean_text'] = text_df['clean_text'].apply(clean_text)

tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1, 2), stop_words='english',
                        min_df=2, max_df=0.95, sublinear_tf=True)
```

```
X_text = tfidf.fit_transform(text_df['clean_text'])
y_text = text_df['is_depression']
```

```
# ========================= TABULAR PREPROCESSING =========================
tab_df = tab_df.copy()
tab_df = tab_df.replace(r'^\s*$', np.nan, regex=True)
tab_df = tab_df.dropna().reset_index(drop=True)
if 'Timestamp' in tab_df.columns:
    tab_df = tab_df.drop('Timestamp', axis=1)

def clean_categorical(col):
    return col.astype(str).str.lower().str.strip()

for col in tab_df.columns:
    if tab_df[col].dtype == 'object':
        tab_df[col] = clean_categorical(tab_df[col])

label_encoders = {}
for col in tab_df.columns:
    if tab_df[col].dtype == 'object':
        le = LabelEncoder()
        tab_df[col] = le.fit_transform(tab_df[col])
        label_encoders[col] = le

target_col = 'Do you have Depression?'
X_tab = tab_df.drop(target_col, axis=1)
y_tab = tab_df[target_col]
feature_columns = X_tab.columns.tolist()

scaler = StandardScaler()
X_tab_scaled = scaler.fit_transform(X_tab)
```

```
# ========================= TRAIN/TEST SPLIT =========================
X_train, X_test, y_train, y_test = train_test_split(X_text, y_text, test_size=0.2, random_state=42, stratify=y_text)
X_train_t, X_test_t, y_train_t, y_test_t = train_test_split(X_tab_scaled, y_tab, test_size=0.2, random_state=42, stratify=y_tat
```

```
# ========================= XGBoost TEXT MODEL =========================
text_model = XGBClassifier(
    scale_pos_weight=len(y_train)/sum(y_train),
    random_state=42,
    n_estimators=300,
    max_depth=6,
    learning_rate=0.1,
    eval_metric='logloss'
)
text_model.fit(X_train, y_train)
pred = text_model.predict(X_test)
print("Text XGBoost Accuracy:", accuracy_score(y_test, pred))
```

```
Text XGBoost Accuracy: 0.9553975436328378
```

```
# ========================= XGBoost TABULAR MODEL =========================
tabular_model = XGBClassifier(
    scale_pos_weight=len(y_train_t)/sum(y_train_t),
    random_state=42,
    n_estimators=200,
    max_depth=5,
    learning_rate=0.1,
    eval_metric='logloss'
)
tabular_model.fit(X_train_t, y_train_t)
pred2 = tabular_model.predict(X_test_t)
print("Tabular XGBoost Accuracy:", accuracy_score(y_test_t, pred2))
```

```
Tabular XGBoost Accuracy: 0.75
```

```
# ========================= AUTOENCODER (Anomaly Detection) =========================
input_dim = X_train_t.shape[1]
input_layer = Input(shape=(input_dim,))
encoded = Dense(16, activation='relu')(input_layer)
encoded = Dense(8, activation='relu')(encoded)
decoded = Dense(16, activation='relu')(encoded)
decoded = Dense(input_dim, activation='linear')(decoded)

autoencoder = Model(inputs=input_layer, outputs=decoded)
autoencoder.compile(optimizer='adam', loss='mse')
autoencoder.fit(X_train_t, X_train_t, epochs=50, batch_size=16, validation_data=(X_test_t, X_test_t), verbose=0)

recon_train = autoencoder.predict(X_train_t, verbose=0)
train_mse = np.mean(np.power(X_train_t - recon_train, 2), axis=1)
threshold = np.percentile(train_mse, 95)
print("Anomaly Threshold:", threshold)
```

```
Anomaly Threshold: 1.2281727375958986
```

```
# ========================= SAVE ALL ARTIFACTS =========================
with open("text_model.pkl", "wb") as f: pickle.dump(text_model, f)
with open("tabular_model.pkl", "wb") as f: pickle.dump(tabular_model, f)
with open("tfidf_vectorizer.pkl", "wb") as f: pickle.dump(tfidf, f)
with open("scaler.pkl", "wb") as f: pickle.dump(scaler, f)
with open("label_encoders.pkl", "wb") as f: pickle.dump(label_encoders, f)
with open("feature_columns.pkl", "wb") as f: pickle.dump(feature_columns, f)
with open("threshold.pkl", "wb") as f: pickle.dump(threshold, f)

autoencoder.save("autoencoder_model.h5")

print("✅ All models and artifacts saved successfully!")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
✅ All models and artifacts saved successfully!
```